

✓ WEEK 1 : Introduction to Machine Learning

- Types of Learning
- Regression vs Classification
- Model Accuracy

```
from google.colab import drive
drive.mount('/content/drive')
```

↗ Drive already mounted at /content/drive; to attempt to forcibly remount, call

```
import os
os.listdir('/content/drive/MyDrive/(SEM2)ML_DATA_FILES')
```

↗ [Show hidden output](#)

✓ Wage data introduced

All data is in the data directory. This data was downloaded from the ISLR and MASS R packages

```
# Bring data into workspace and replicate plots
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

```
df_wage = pd.read_csv('/content/drive/MyDrive/(SEM2)ML_DATA_FILES/wage.csv')
```

```
df_wage.head(4)
```



	year	age	sex	maritl	race	education	region	jobclass	health	health_
0	2006	18	1. Male	1. Never Married	1. White	1. < HS Grad	2. Middle Atlantic	1. Industrial	1. <=Good	
1	2004	24	1. Male	1. Never Married	1. White	4. College Grad	2. Middle Atlantic	2. Information	2. >=Very Good	
2	2003	45	1. Male	2. Married	1. White	3. Some College	2. Middle Atlantic	1. Industrial	1. <=Good	1

Next steps:

[Generate code with df_wage](#)

[View recommended plots](#)

[New interactive sheet](#)

```
df_wage.info()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   year            3000 non-null   int64
1   age             3000 non-null   int64
2   sex             3000 non-null   object
3   maritl          3000 non-null   object
4   race            3000 non-null   object
5   education       3000 non-null   object
6   region          3000 non-null   object
7   jobclass        3000 non-null   object
8   health          3000 non-null   object
9   health_ins      3000 non-null   object
10  logwage         3000 non-null   float64
11  wage            3000 non-null   float64
dtypes: float64(2), int64(2), object(8)
memory usage: 281.4+ KB
```

```
df_wage.describe(include=['object'])
```



	sex	maritl	race	education	region	jobclass	health	health_ins
count	3000	3000	3000	3000	3000	3000	3000	3000
unique	1	5	4	5	1	2	2	2
top	1. Male	2. Married	1. White	2. HS Grad	2. Middle Atlantic	1. Industrial	2. >=Very Good	1. Yes

```
# Look at all available plotting styles
plt.style.available
```

```
➦ ['Solarize_Light2',
  '_classic_test_patch',
  '_mpl-gallery',
  '_mpl-gallery-nogrid',
  'bmh',
  'classic',
  'dark_background',
  'fast',
  'fivethirtyeight',
  'ggplot',
  'grayscale',
  'petroff10',
  'seaborn-v0_8',
  'seaborn-v0_8-bright',
  'seaborn-v0_8-colorblind',
  'seaborn-v0_8-dark',
  'seaborn-v0_8-dark-palette',
  'seaborn-v0_8-darkgrid',
  'seaborn-v0_8-deep',
  'seaborn-v0_8-muted',
  'seaborn-v0_8-notebook',
  'seaborn-v0_8-paper',
  'seaborn-v0_8-pastel',
  'seaborn-v0_8-poster',
  'seaborn-v0_8-talk',
  'seaborn-v0_8-ticks',
  'seaborn-v0_8-white',
  'seaborn-v0_8-whitegrid',
  'tableau-colorblind10']
```

```
plt.style.use("ggplot")
```

```
# need to reshape data to plot correctly
df_edu = df_wage.pivot(columns='education', values='wage')
```

```
df_edu.head(8)
```



education	1. < HS Grad	2. HS Grad	3. Some College	4. College Grad	5. Advanced Degree
0	75.043154	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	70.476020	NaN
2	NaN	NaN	130.982177	NaN	NaN
3	NaN	NaN	NaN	154.685293	NaN
4	NaN	75.043154	NaN	NaN	NaN
5	NaN	NaN	NaN	127.115744	NaN
6	NaN	NaN	169.528538	NaN	NaN
7	NaN	NaN	111.720849	NaN	NaN



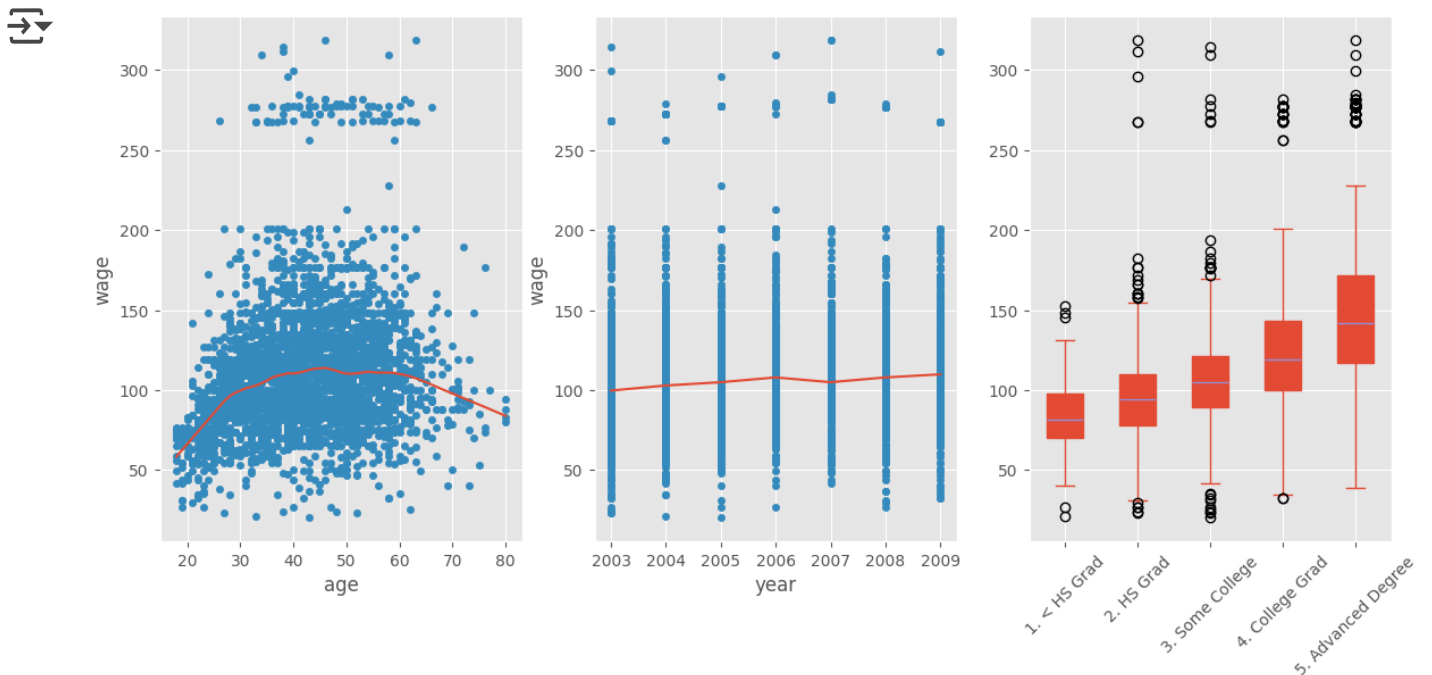
Next steps: [View recommended plots](#) [New interactive sheet](#)

```
# importing statsmodels library to fit lowess curve through data
import statsmodels.api as sm
```

```
fig, ax = plt.subplots(1, 3, figsize=(14,6))
df_wage.plot.scatter('age', 'wage', ax=ax[0])
lowess = sm.nonparametric.lowess(df_wage['wage'], df_wage['age'], frac=.2)
ax[0].plot(lowess[:, 0], lowess[:, 1])

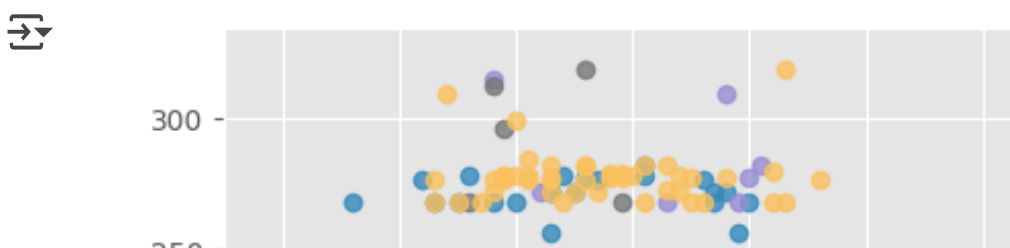
df_wage.plot.scatter('year', 'wage', ax=ax[1])
year_median = df_wage.groupby('year')['wage'].median()
ax[1].plot(year_median)

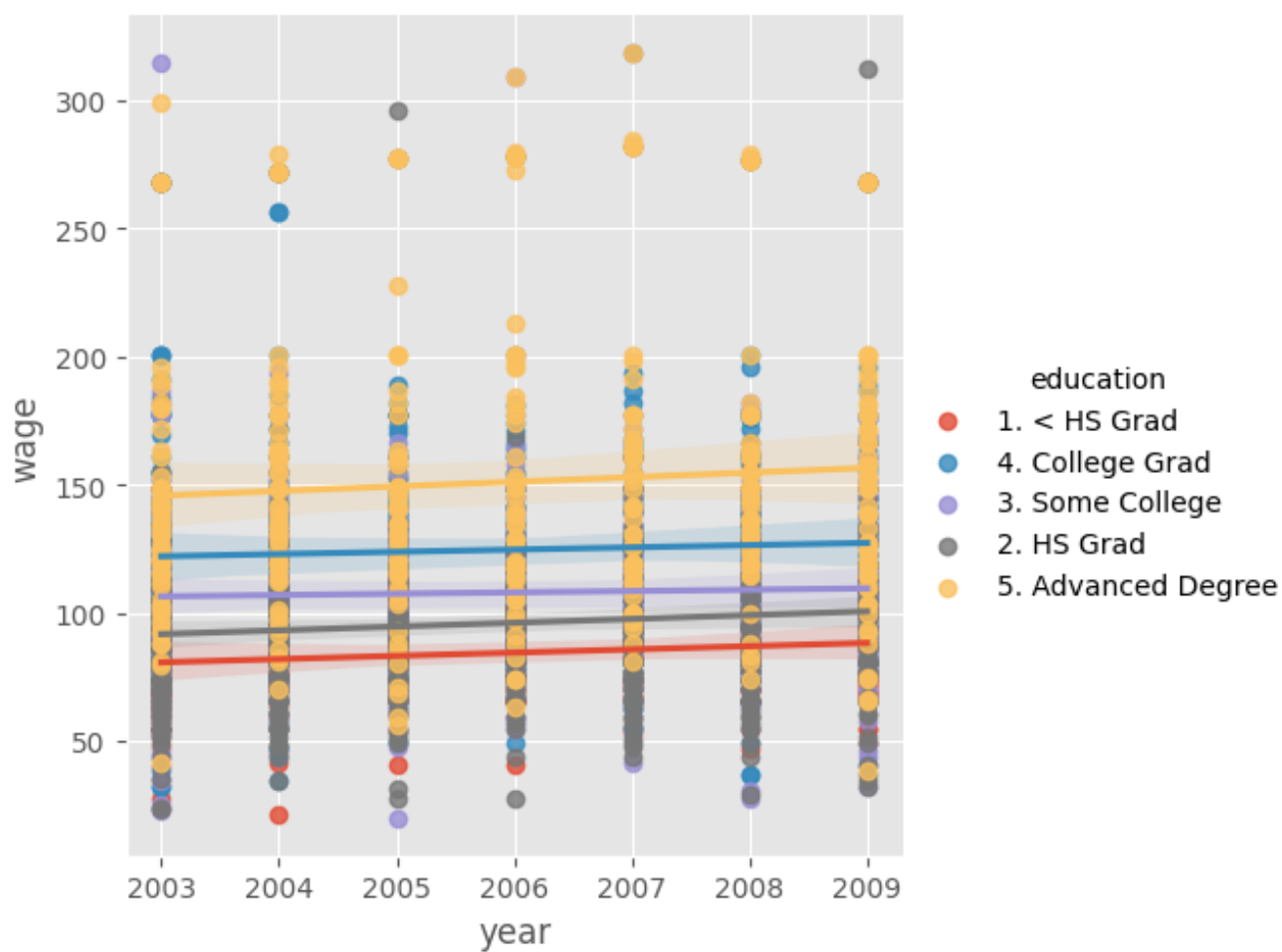
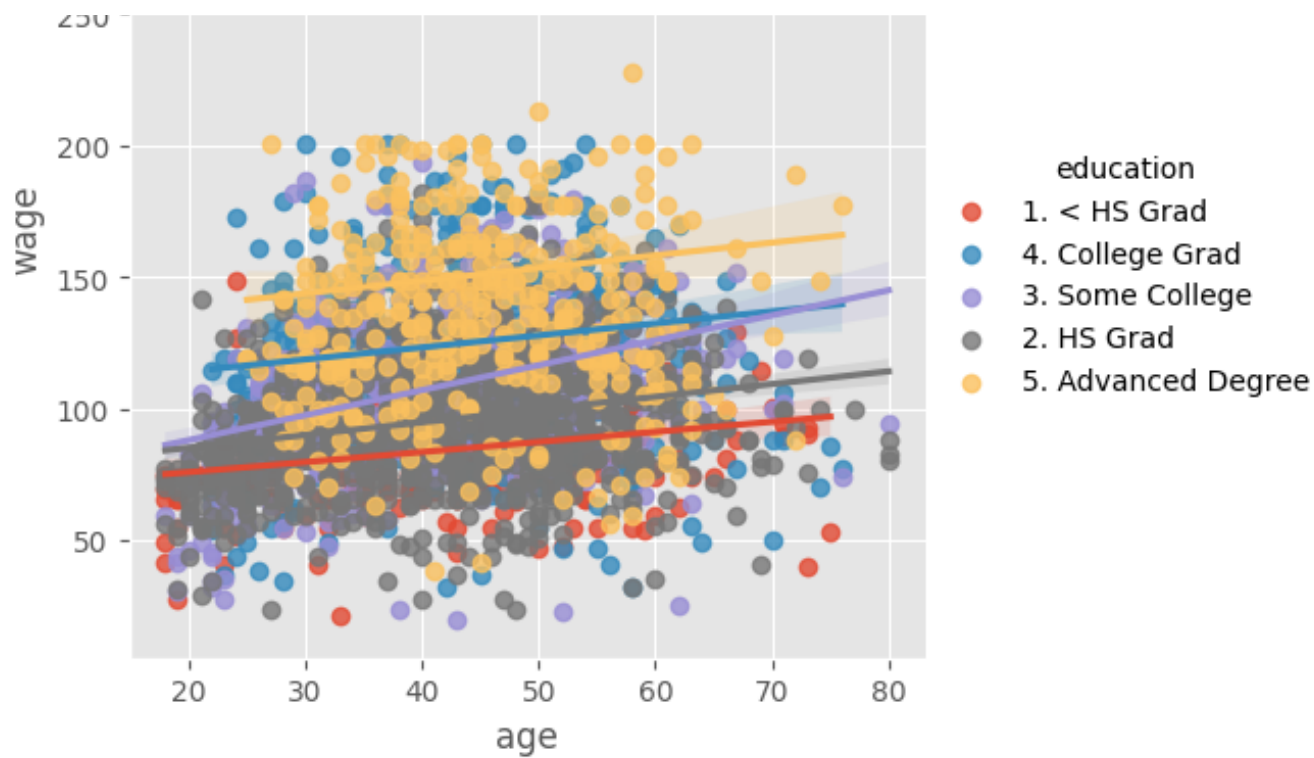
boxplot = df_edu.plot.box(ax=ax[2], rot=45, patch_artist=True)
colors = ['lightblue', 'green', 'yellow', 'blue', 'red']
for artist, color in zip(boxplot.artists, colors):
    artist.set_facecolor(color)
```



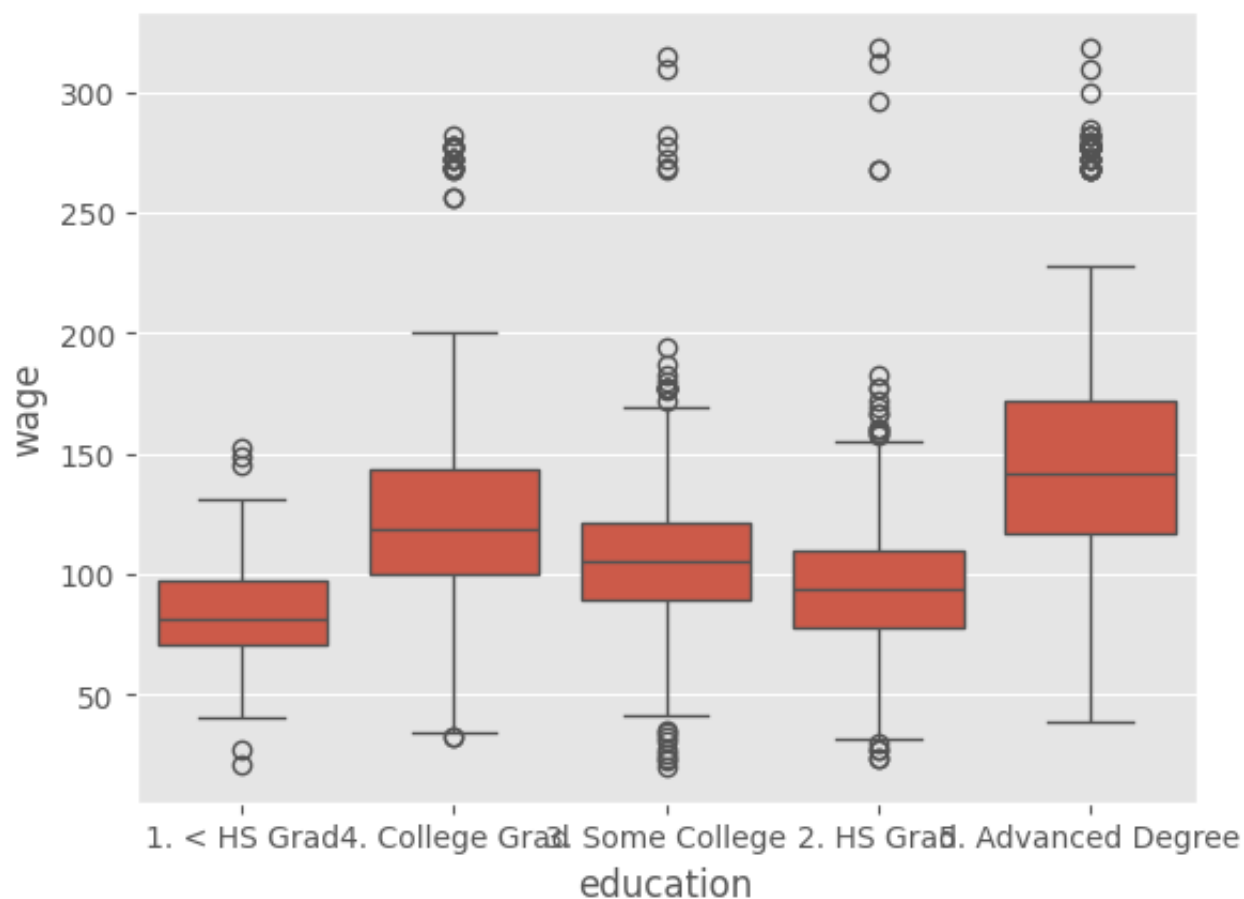
```
import seaborn as sns
```

```
# Similar plots to those above
sns.lmplot(x='age', y='wage', data=df_wage, hue='education')
sns.lmplot(x='year', y='wage', data=df_wage, ci=99.99, hue='education');
```





```
sns.boxplot(x='education', y='wage', data=df_wage);
```



```
df1 = df_wage[['age', 'year', 'education', 'wage']]
df1.head()
```



	age	year	education	wage
0	18	2006	1. < HS Grad	75.043154
1	24	2004	4. College Grad	70.476020
2	45	2003	3. Some College	130.982177
3	43	2003	4. College Grad	154.685293
4	50	2005	2. HS Grad	75.043154



Next steps:

[View recommended plots](#)

[New interactive sheet](#)

```
df_melt = pd.melt(df1, id_vars=['education', 'wage'])
```

```
df_melt.head()
```



	education	wage	variable	value
0	1. < HS Grad	75.043154	age	18
1	4. College Grad	70.476020	age	24
2	3. Some College	130.982177	age	45
3	4. College Grad	154.685293	age	43
4	2. HS Grad	75.043154	age	50



Next
steps:

[Generate code with df_melt](#)

[View recommended plots](#)

[New interactive sheet](#)

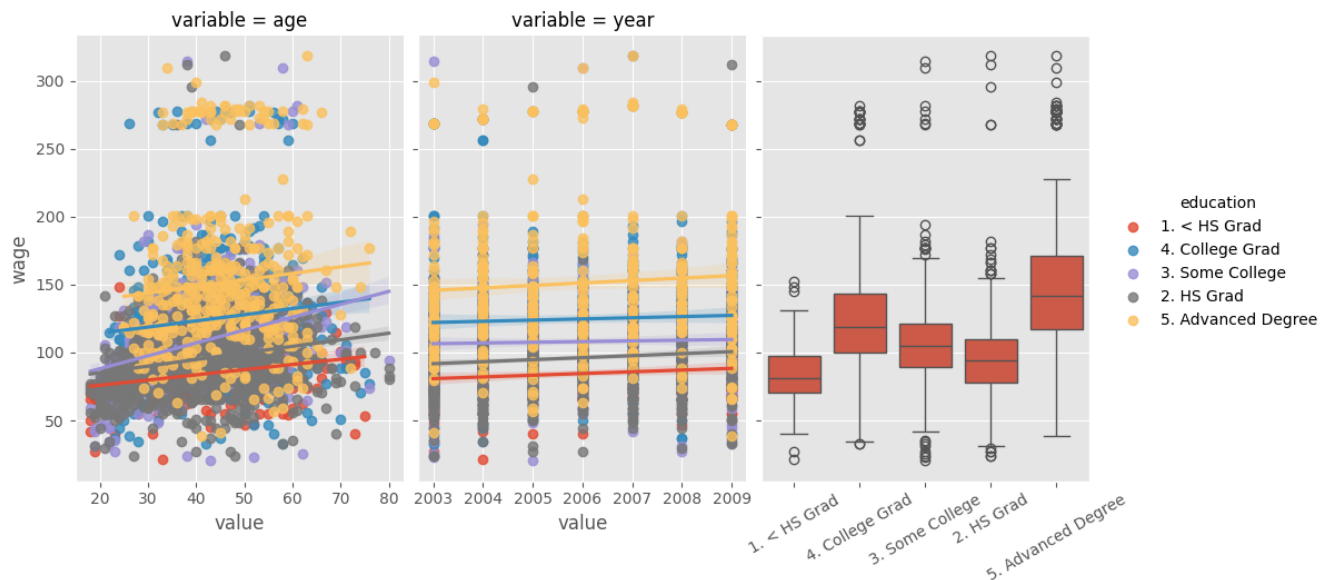
```
seaborn_grid = sns.lmplot(x='value', y='wage', col='variable', hue='education', data=df_melt)
seaborn_grid.fig.set_figwidth(8)
```

```
left, bottom, width, height = seaborn_grid.fig.axes[0]._position.bounds
left2, bottom2, width2, height2 = seaborn_grid.fig.axes[1]._position.bounds
left_diff = left2 - left
seaborn_grid.fig.add_axes((left2 + left_diff, bottom, width, height))
```

```
sns.boxplot(x='education', y='wage', data=df_wage, ax = seaborn_grid.fig.axes[2])
ax2 = seaborn_grid.fig.axes[2]
ax2.set_yticklabels([])
ax2.set_xticklabels(ax2.get_xmajorticklabels(), rotation=30)
ax2.set_ylabel('')
ax2.set_xlabel('');
```

```
leg = seaborn_grid.fig.legends[0]
leg.set_bbox_to_anchor([0, .1, 1.5,1])
```

```
➡ /usr/local/lib/python3.11/dist-packages/seaborn/regression.py:598: UserWarning:
  warnings.warn(msg, UserWarning)
<ipython-input-29-6cbdc0e04ec1>:12: UserWarning: set_ticklabels() should on
  ax2.set_xticklabels(ax2.get_xmajorticklabels(), rotation=30)
```



✓ Regression vs Classification

```
df_smarket = pd.read_csv('/content/drive/MyDrive/(SEM2)ML_DATA_FILES/smarket.csv')
```

```
df_smarket.head()
```



	Year	Lag1	Lag2	Lag3	Lag4	Lag5	Volume	Today	Direction
0	2001	0.381	-0.192	-2.624	-1.055	5.010	1.1913	0.959	Up
1	2001	0.959	0.381	-0.192	-2.624	-1.055	1.2965	1.032	Up
2	2001	1.032	0.959	0.381	-0.192	-2.624	1.4112	-0.623	Down
3	2001	-0.623	1.032	0.959	0.381	-0.192	1.2760	0.614	Up
4	2001	0.614	-0.623	1.032	0.959	0.381	1.2057	0.213	Up



Next
steps:

[Generate code with df_smarket](#)[View recommended plots](#)[New interactive sheet](#)

```
# Put all lags in one column. Make 'Tidy' Data
df_smarket_pivot = pd.melt(df_smarket,
                           id_vars='Direction',
                           value_vars=['Lag1', 'Lag2', 'Lag3'],
                           var_name='Lag Type',
                           value_name='Pct Change')
```

```
df_smarket_pivot.head()
```



	Direction	Lag Type	Pct Change
0	Up	Lag1	0.381
1	Up	Lag1	0.959
2	Down	Lag1	1.032
3	Up	Lag1	-0.623
4	Up	Lag1	0.614

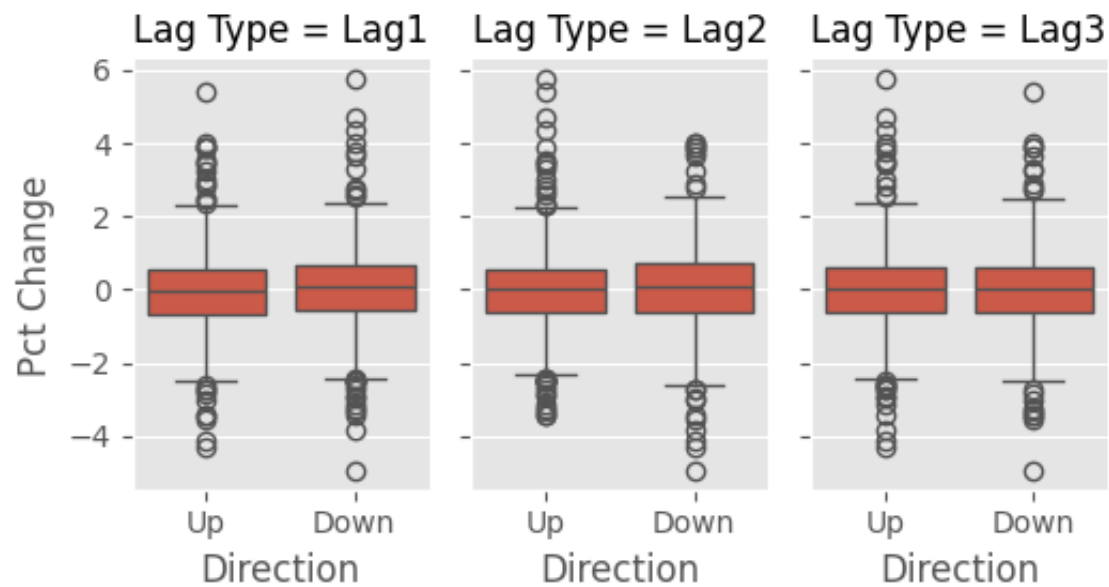


Next
steps:

[Generate code with df_smarket_pivot](#)[View recommended plots](#)[New interactive](#)

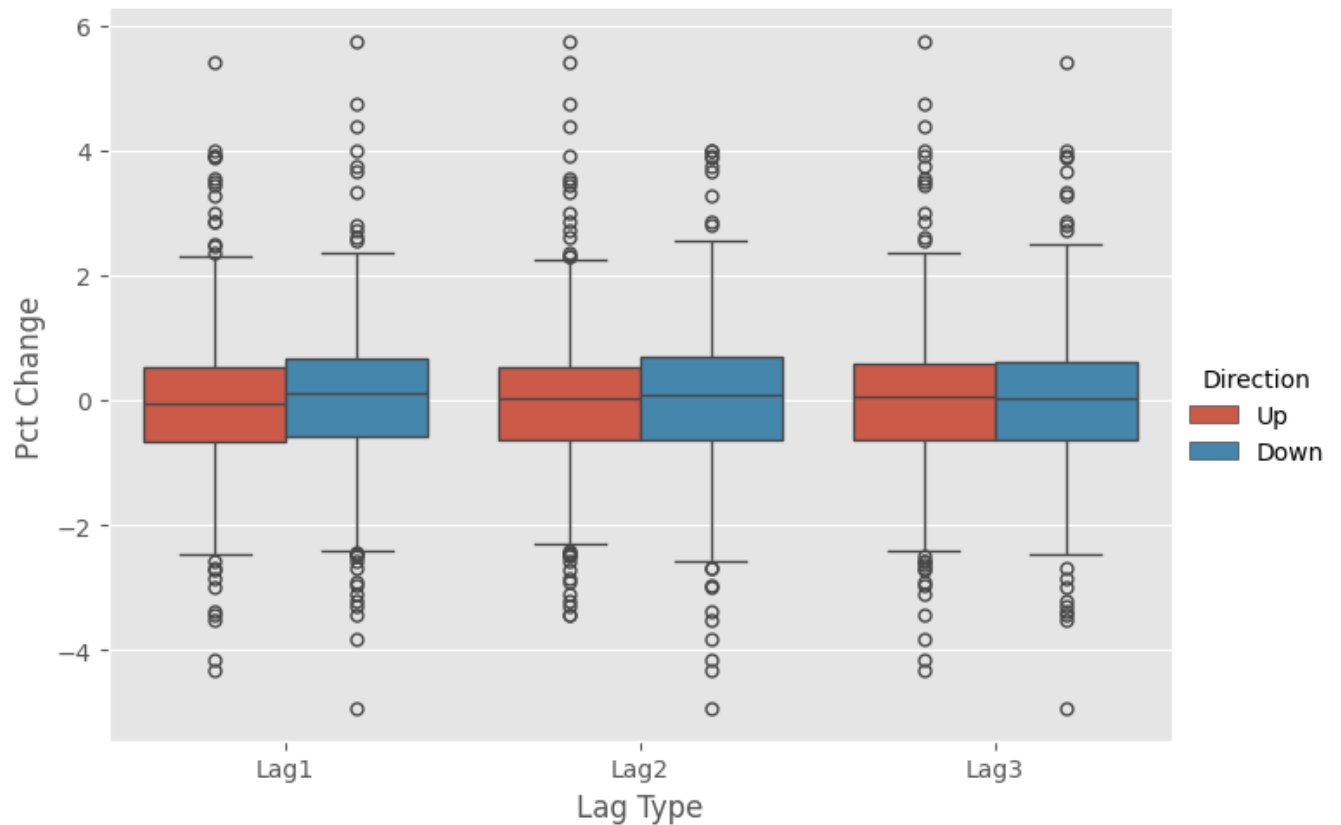
```
g = sns.FacetGrid(df_smarket_pivot, col="Lag Type", aspect=.6)
g = g.map(sns.boxplot, "Direction", "Pct Change")
```

 /usr/local/lib/python3.11/dist-packages/seaborn/axisgrid.py:718: UserWarning: warnings.warn(warning)



```
sns.catplot(x="Lag Type", y="Pct Change", hue="Direction", data=df_smarket_pivot, kind="box",
```

```
<seaborn.axisgrid.FacetGrid at 0x7e7221c44090>
```



✓ Clustering

No longer interested in prediction - looking to discover underlying similarities in the data

```
df_genes = pd.read_csv('/content/drive/MyDrive/(SEM2)ML_DATA_FILES/nci60_data.csv', index_c
```

```
df_genes.head()
```



	1	2	3	4	5	6	7	
V1	0.300000	1.180000	0.550000	1.140000	-0.265000	-7.000000e-02	0.350000	-0.315
V2	0.679961	1.289961	0.169961	0.379961	0.464961	5.799610e-01	0.699961	0.724
V3	0.940000	-0.040000	-0.170000	-0.040000	-0.605000	0.000000e+00	0.090000	0.645
V4	0.280000	-0.310000	0.680000	-0.810000	0.625000	-1.387779e-17	0.170000	0.245
V5	0.485000	-0.465000	0.395000	0.905000	0.200000	-5.000000e-03	0.085000	0.110

5 rows × 6830 columns

```
from sklearn.decomposition import PCA
```

```
pca = PCA(2)
```

```
df_genes_transformed = pd.DataFrame(pca.fit_transform(df_genes), columns=['PC1', 'PC2'])
```

```
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(4)
```

```
kmeans.fit(df_genes_transformed)
```



```
KMeans
KMeans(n_clusters=4)
```

```
kmeans.labels_
```

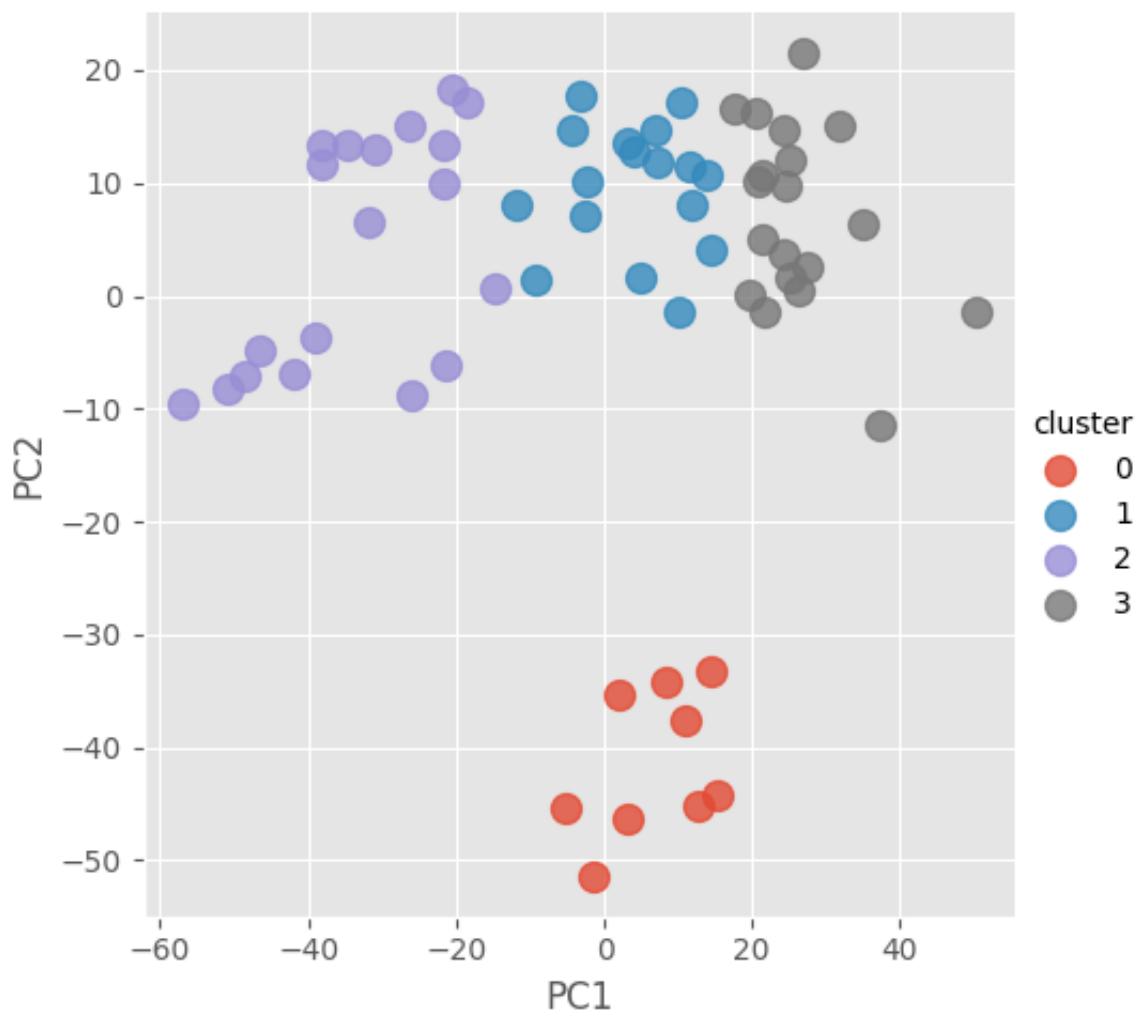


```
array([3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 1, 3, 1, 1, 3,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 2, 0, 0, 0, 0, 0, 0, 0, 0],
      dtype=int32)
```

```
df_genes_transformed['cluster'] = kmeans.labels_
```

```
sns.lmplot(x='PC1', y='PC2', data=df_genes_transformed, fit_reg=False, hue='cluster',  
          scatter_kws={"marker": "D", "s": 100})
```

 <seaborn.axisgrid.FacetGrid at 0x7e7221ecf710>



✓ Advertising Data

The advertising data consists of product sales from 200 markets and their associated tv, radio, and newspaper advertising budgets. What kind of relationship can be seen between advertising budget and sales

```
df_adv = pd.read_csv('/content/drive/MyDrive/(SEM2)ML_DATA_FILES/Advertising.csv')
df_adv.head(10)
```



	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9
5	8.7	48.9	75.0	7.2
6	57.5	32.8	23.5	11.8
7	120.2	19.6	11.6	13.2
8	8.6	2.1	1.0	4.8
9	199.8	2.6	21.2	10.6



Next steps:

[Generate code with df_adv](#)

[View recommended plots](#)

[New interactive sheet](#)

```
df_adv_new = pd.melt(df_adv, value_vars=['TV', 'Radio', 'Newspaper'], id_vars='Sales', value
df_adv_new.head(10)
```



	Sales	variable	adv_budget
0	22.1	TV	230.1
1	10.4	TV	44.5
2	9.3	TV	17.2
3	18.5	TV	151.5
4	12.9	TV	180.8
5	7.2	TV	8.7
6	11.8	TV	57.5
7	13.2	TV	120.2
8	4.8	TV	8.6
9	10.6	TV	199.8



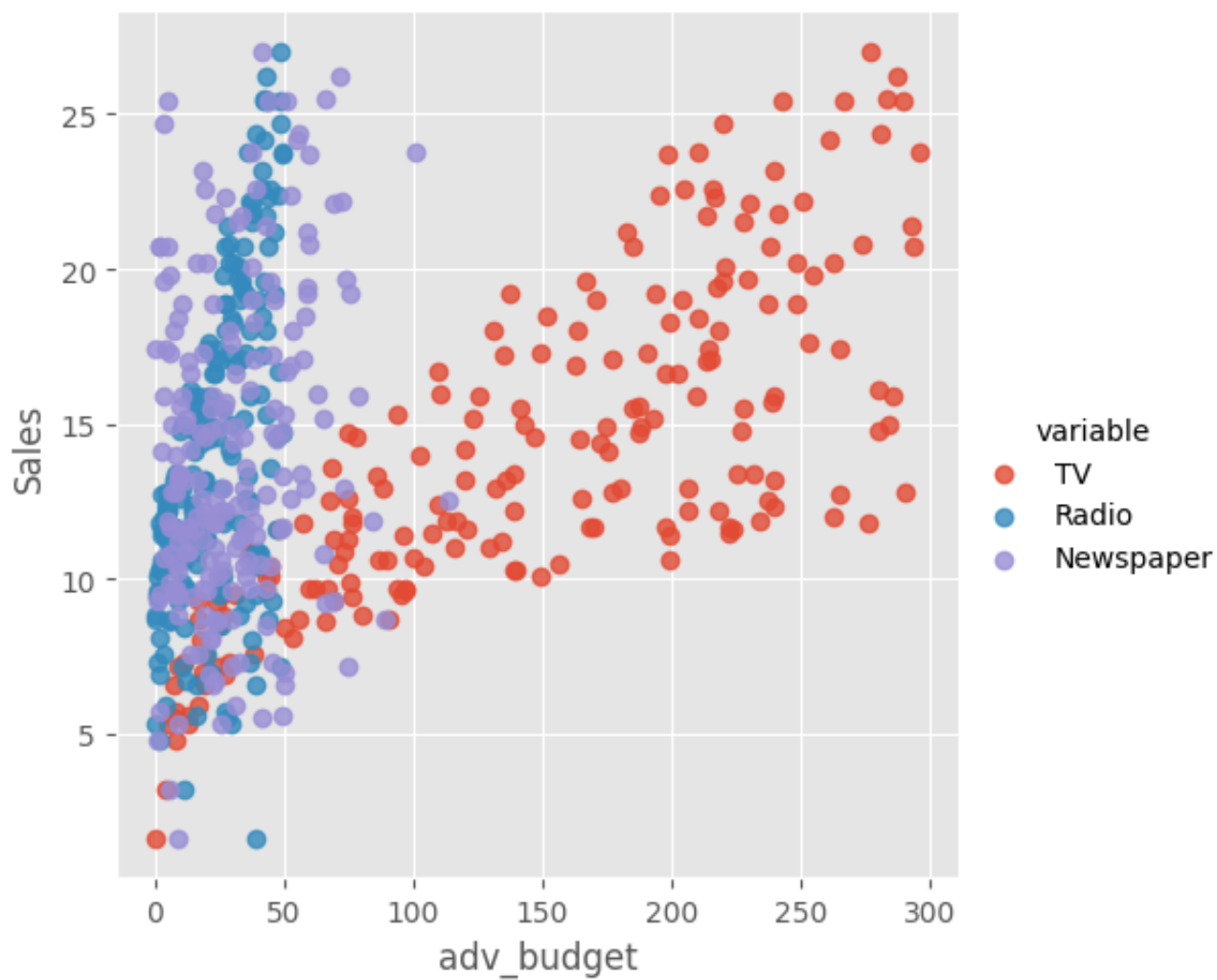
Next steps:

[Generate code with df_adv_new](#)

[View recommended plots](#)

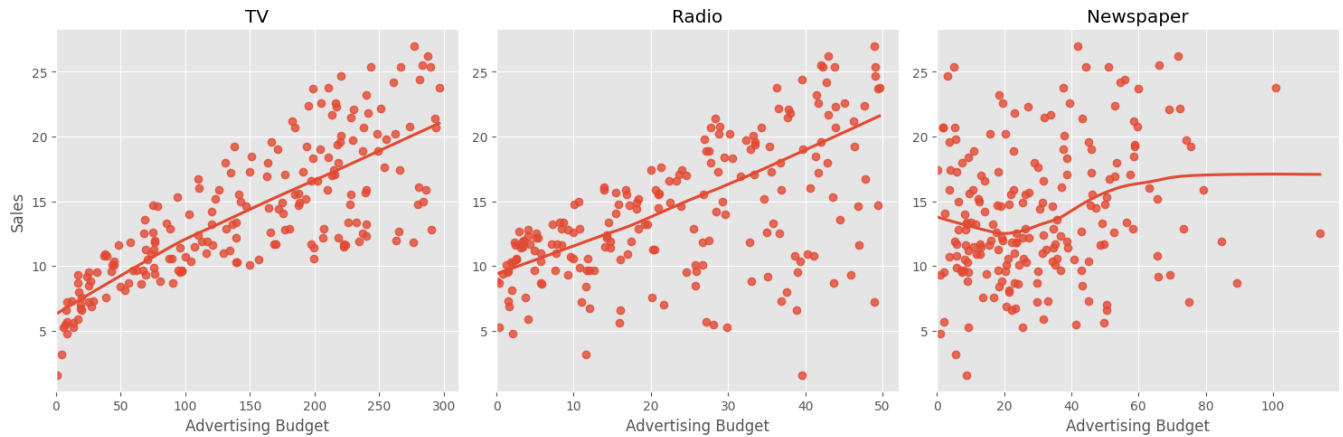
[New interactive sheet](#)

```
sns.lmplot(x='adv_budget', y='Sales', data=df_adv_new, hue='variable', fit_reg=False);
```



```
lm = sns.lmplot(x='adv_budget', y='Sales', data=df_adv_new, col='variable', sharey=False,
               axes = lm.axes)
for i, ax in enumerate(axes[0]):
    ax.set_xlim(0,)
    ax.set_title(lm.col_names[i])
    ax.set_xlabel('Advertising Budget')
```

↔ /usr/local/lib/python3.11/dist-packages/seaborn/regression.py:598: UserWarning
 warnings.warn(msg, UserWarning)
 /usr/local/lib/python3.11/dist-packages/seaborn/regression.py:598: UserWarning
 warnings.warn(msg, UserWarning)



✓ Training Data vs Testing Data

Training Data - Data used to build a prediction model. Should not be used to validate the model.

Testing Data - Data used to determine the usefulness of the model. Validates the model. This data is unseen during model building phase.

```
num_points = 30
```

```
np.random.seed(12345)
x = np.linspace(1,13, num_points).reshape(-1, 1)
error = np.random.randn(num_points, 1) * num_points
f = lambda x: (x - 2) * (x - 6) * (x - 12)

y = f(x) + error
```

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

```
poly = PolynomialFeatures(degree=10)
X = poly.fit_transform(x)
```

```
obs_nums = np.arange(0, num_points)
np.random.shuffle(obs_nums)

top_70 = int(num_points * .7)
rand_train = np.sort(obs_nums[:top_70])
rand_test = np.sort(obs_nums[top_70:])
```

```
X_train = X[rand_train]
X_test = X[rand_test]
y_train = y[rand_train]
y_test = y[rand_test]
```

```
linreg = LinearRegression()
linreg.fit(X_train[:, :2], y_train)
y_train_2 = linreg.predict(X_train[:, :2])
y_test_2 = linreg.predict(X_test[:, :2])

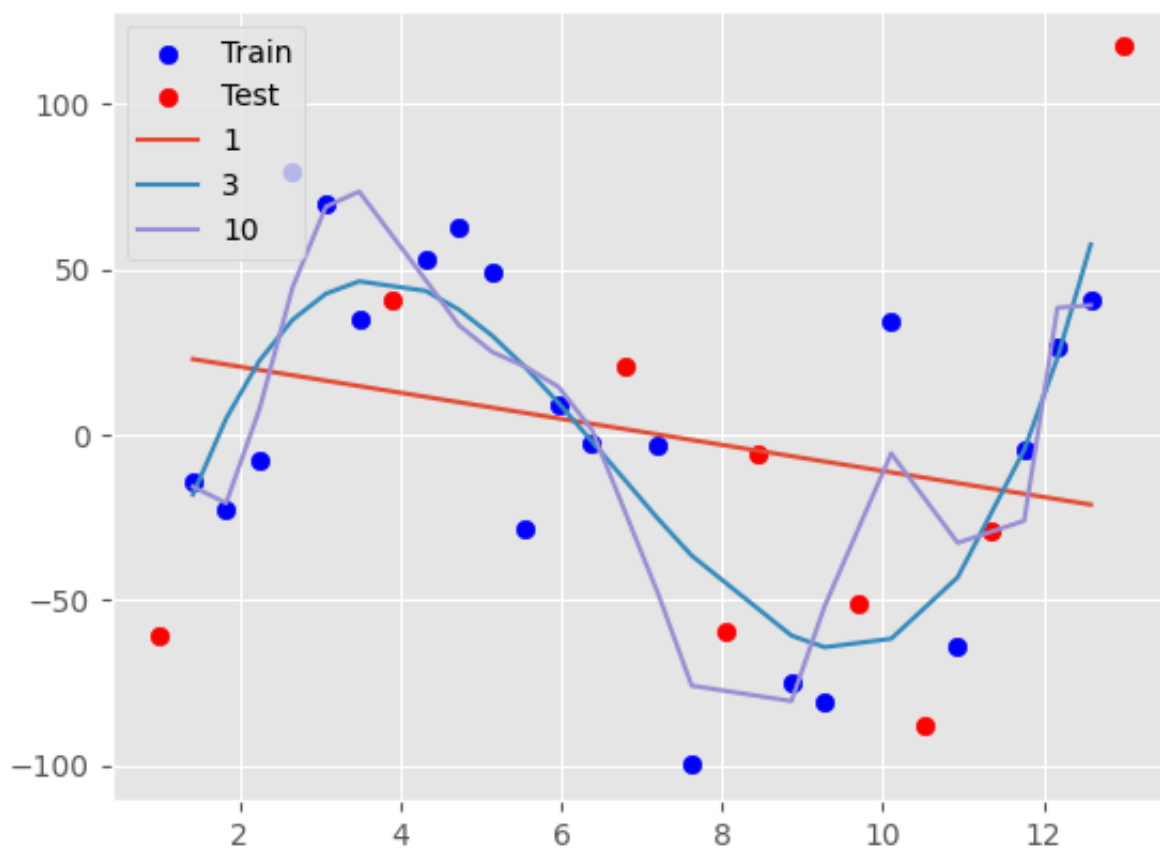
linreg.fit(X_train[:, :4], y_train)
y_train_4 = linreg.predict(X_train[:, :4])
y_test_4 = linreg.predict(X_test[:, :4])

linreg.fit(X_train, y_train)
y_train_10 = linreg.predict(X_train)
y_test_10 = linreg.predict(X_test)
```

```
errors_train= np.array([np.mean((y_train - y_train_2) ** 2),
                        np.mean((y_train - y_train_4) ** 2),
                        np.mean((y_train - y_train_10) ** 2)])
errors_train = np.column_stack([2, 4, 10], errors_train))

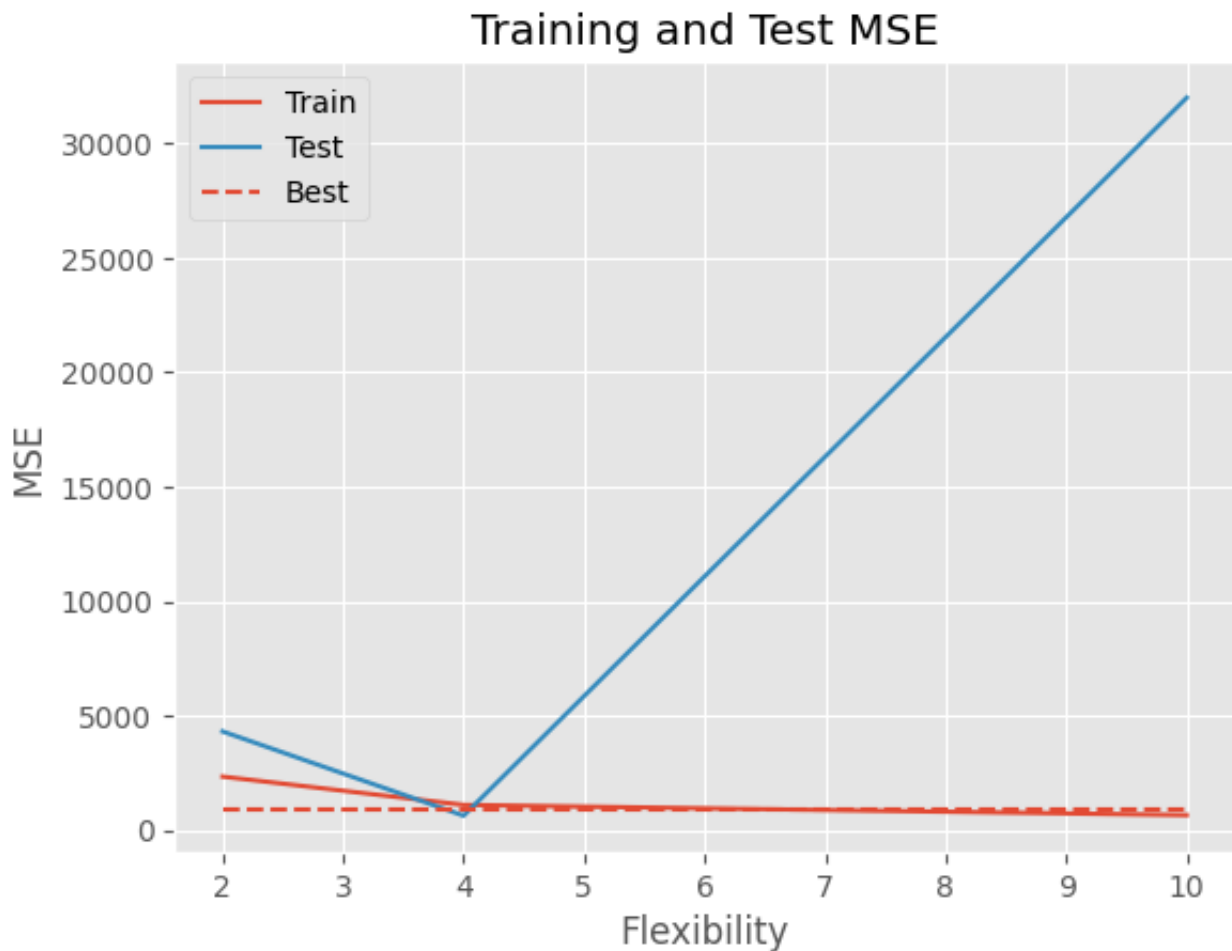
errors_test = np.array([np.mean((y_test - y_test_2) ** 2),
                        np.mean((y_test - y_test_4) ** 2),
                        np.mean((y_test - y_test_10) ** 2)])
errors_test = np.column_stack([2, 4, 10], errors_test))
```

```
plt.scatter(X_train[:,1], y_train, c='b', label='Train')
plt.scatter(X_test[:,1], y_test, c='r', label = 'Test')
plt.plot(X_train[:,1], y_train_2, label = '1')
plt.plot(X_train[:,1], y_train_4, label = '3')
plt.plot(X_train[:,1], y_train_10, label = '10')
plt.legend(loc=2);
```



```
plt.plot(errors_train[:, 0], errors_train[:, 1], label = 'Train')
plt.plot(errors_test[:, 0], errors_test[:, 1], label = 'Test')
plt.hlines(900, 2, 10, label = 'Best', linestyle = '--')

plt.legend()
plt.title("Training and Test MSE")
plt.xlabel('Flexibility')
plt.ylabel('MSE');
```



Problem (advanced)

Write a function that takes a list/array of how many parameters to fit a linear regression model for the above data and outputs the two plots above.

Exercises

✓ Problem 3

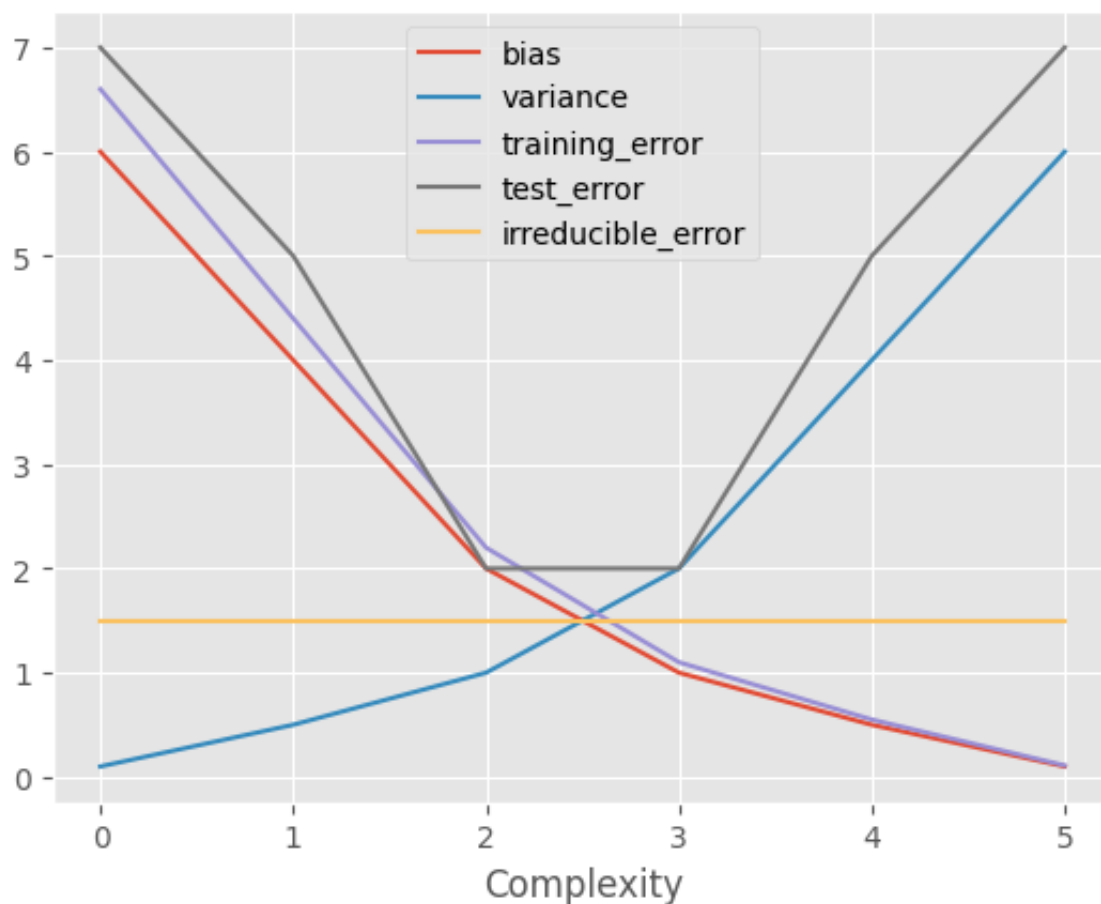
3a. Hand-picked points to show the 5 curves

```
bias = np.array([6, 4, 2, 1, .5, .1])
variance = bias[::-1]
training_error = bias * 1.1
test_error = np.array([7, 5, 2, 2, 5, 7])
irreducible_error = np.ones(6) * 1.5
```

```
df_3 = pd.DataFrame({'bias': bias,
                    'variance': variance,
                    'training_error': training_error,
                    'test_error': test_error,
                    'irreducible_error': irreducible_error})
```

```
df_3.plot()
plt.xlabel('Complexity')
```

↔ Text(0.5, 0, 'Complexity')



✓ Problem 7

```
df_7 = pd.DataFrame({'x1': [0, 2, 0, 0, -1, 1], 'x2': [3, 0, 1, 1, 0, 1], 'x3': [0, 0, 3, 2, 1, 1],  
                    'y': ['R', 'R', 'R', 'G', 'G', 'R']})
```

df_7



	x1	x2	x3	y
0	0	3	0	R
1	2	0	0	R
2	0	1	3	R
3	0	1	2	G
4	-1	0	1	G
5	1	1	1	R



Next steps:

[Generate code with df_7](#)[View recommended plots](#)[New interactive sheet](#)

```
# PART a  
# Get x1, x2, x3 from the above dataframe. Subtract (0, 0, 0) from it and square each dimension  
dist = (df_7.values[:, :3] - np.array([0, 0, 0])) ** 2  
  
# Sum across the rows and make sure the type is float  
summed_distance = dist.sum(axis=1).astype('float')  
  
# Take square root to get euclidean distance  
euclidean_dist = np.sqrt(summed_distance)  
euclidean_dist
```



```
array([3.16227766, 2.23606798, 3.16227766, 2.23606798, 1.41421356,  
       1.73205081])
```

✓ Problem 8

✓ Part a and b

```
df_college = pd.read_csv('/content/drive/MyDrive/(SEM2)ML_DATA_FILES/college.csv', index_col=0)
df_college.head()
```

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P
Abilene Christian University	Yes	1660	1232	721	23	52	2885	
Adelphi University	Yes	2186	1924	512	16	29	2683	
Adrian College	Yes	1428	1097	336	22	50	1036	
Agnes Scott College	Yes	417	349	137	60	89	510	
Alaska Pacific University	Yes	193	146	55	16	44	249	

Next steps:

[Generate code with df_college](#)

[View recommended plots](#)

[New interactive sheet](#)

Part c

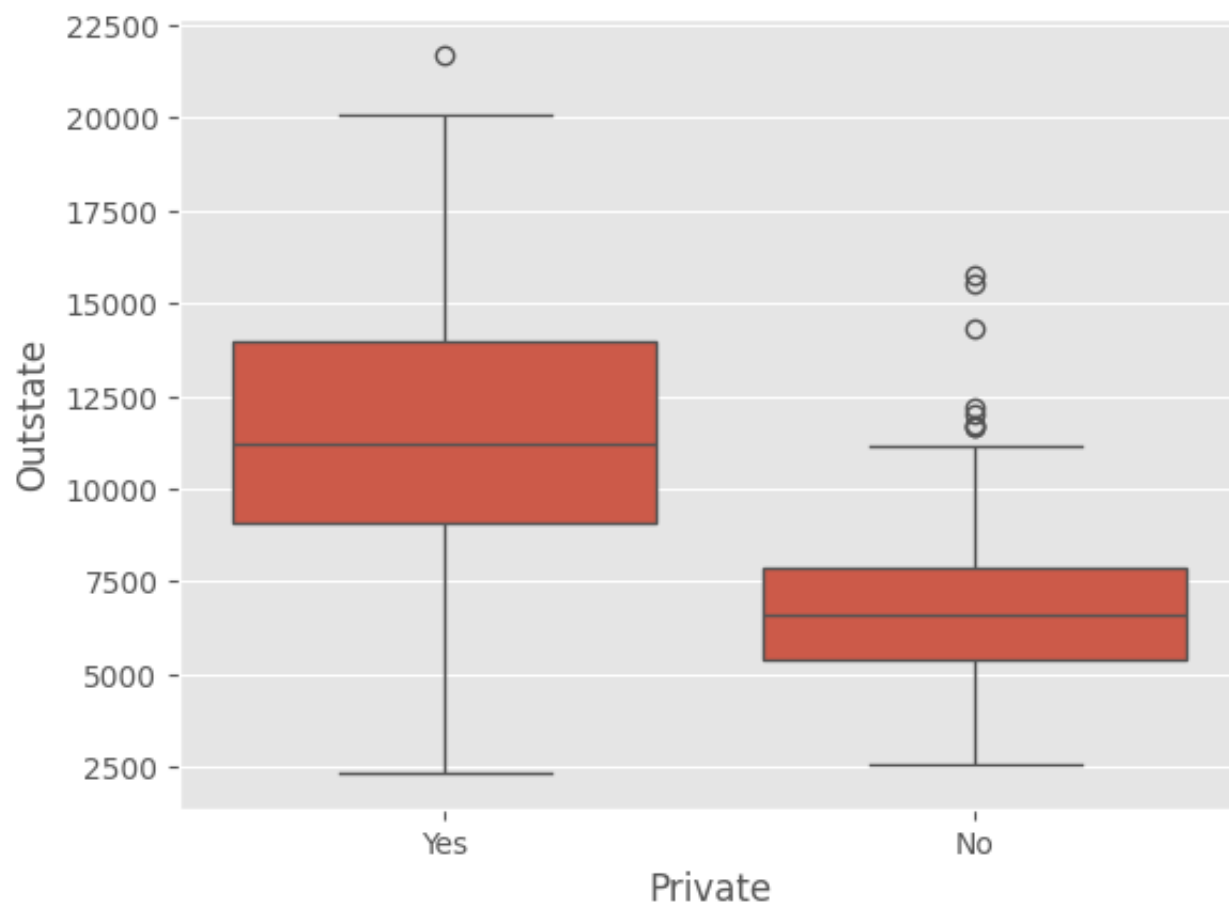
```
# i
df_college.describe()
```

	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad
count	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000
mean	3001.638353	2018.804376	779.972973	27.558559	55.796654	3699.907336
std	3870.201484	2451.113971	929.176190	17.640364	19.804778	4850.420531
min	81.000000	72.000000	35.000000	1.000000	9.000000	139.000000
25%	776.000000	604.000000	242.000000	15.000000	41.000000	992.000000
50%	1558.000000	1110.000000	434.000000	23.000000	54.000000	1707.000000
75%	3624.000000	2424.000000	902.000000	35.000000	69.000000	4005.000000
max	48094.000000	26330.000000	6392.000000	96.000000	100.000000	31643.000000

```
# ii
sns.pairplot(df_college.iloc[:, :10]);
```



```
# iii
sns.boxplot(x='Private', y='Outstate', data=df_college);
```



df_college



	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad
Abilene Christian University	Yes	1660	1232	721	23	52	2885
Adelphi University	Yes	2186	1924	512	16	29	2685
Adrian College	Yes	1428	1097	336	22	50	1030
Agnes Scott College	Yes	417	349	137	60	89	510
Alaska Pacific University	Yes	193	146	55	16	44	245
...	...	...	...	...	...	...	...
Worcester State College	No	2197	1515	543	4	26	3085
Xavier University	Yes	1959	1805	695	24	47	2845
Xavier University of Louisiana	Yes	2097	1915	695	34	61	2795
Yale University	Yes	10705	2453	1317	95	99	5215
York College of Pennsylvania	Yes	2989	1855	691	28	63	2985

777 rows × 18 columns

Next steps:

[Generate code with df_college](#)

[View recommended plots](#)

[New interactive sheet](#)

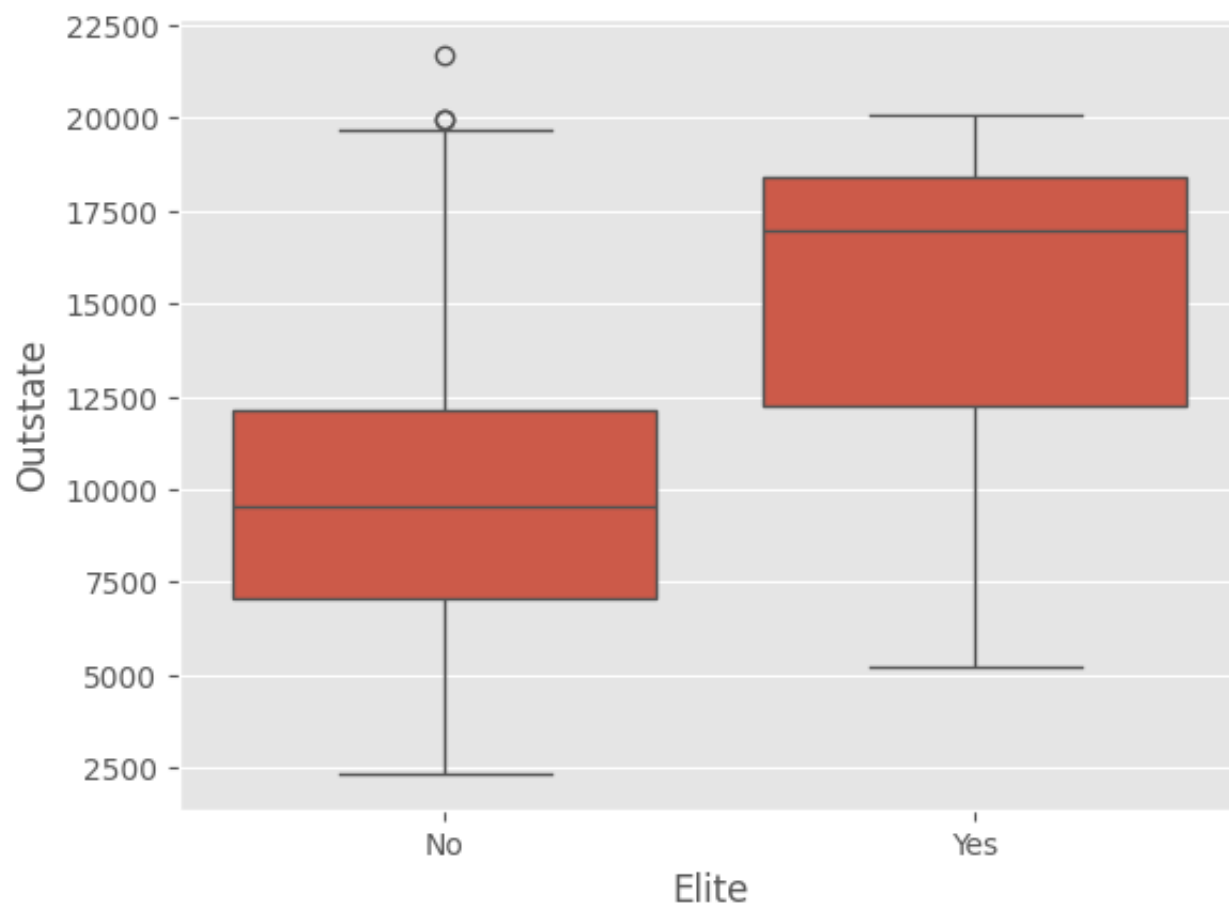
```
# iv
# Next line produces No/Yes categories based on a boolean(0/1) and saves it as a DataFrame c
df_college['Elite'] = pd.Categorical(np.where(df_college['Top10perc'] > 50, 'Yes', 'No'))
print(df_college['Elite'].value_counts())
sns.boxplot(x='Elite', y='Outstate', data=df_college);
```

Elite

No 699

Yes 78

Name: count, dtype: int64

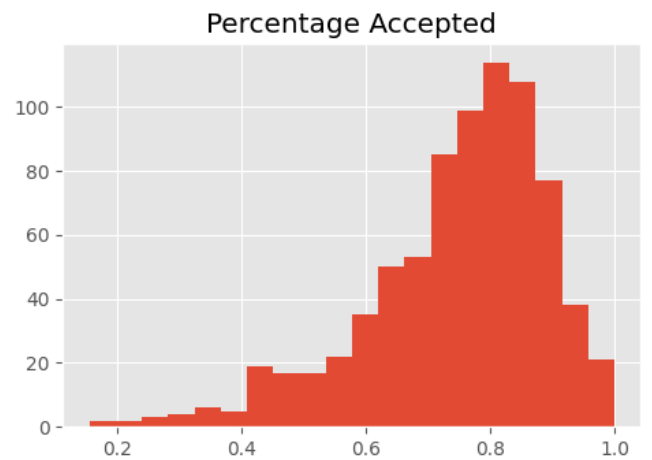
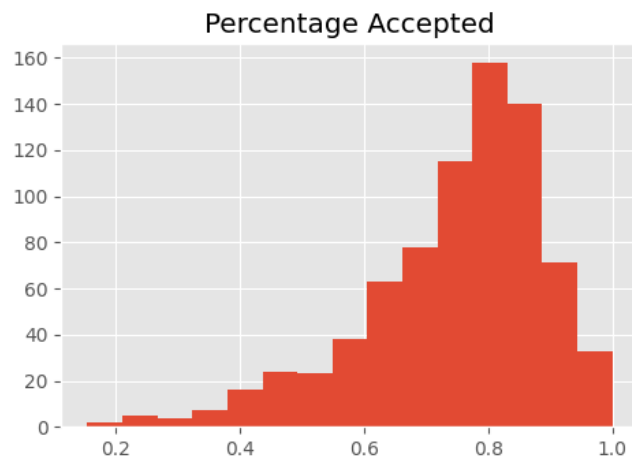
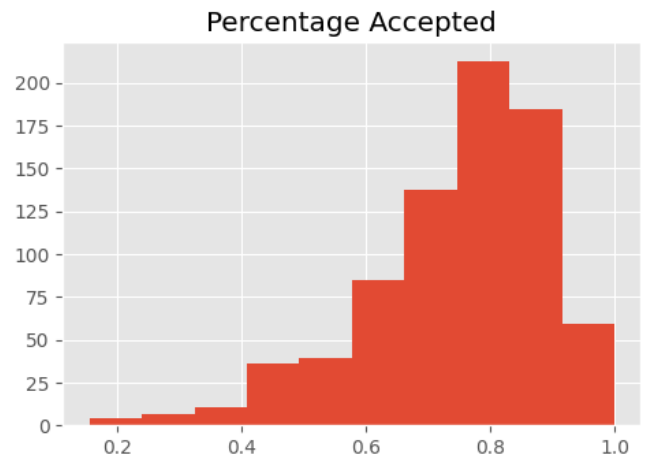
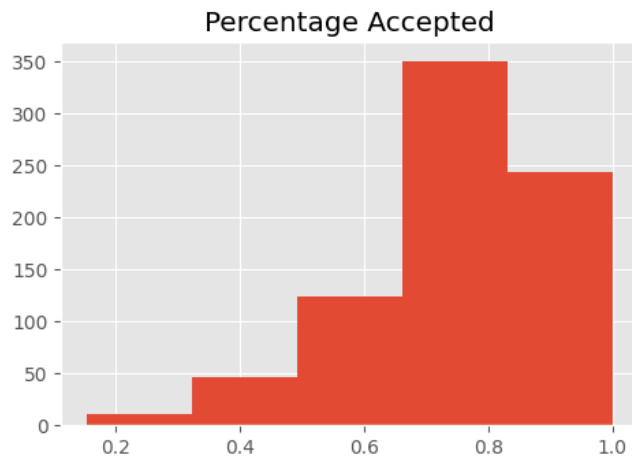


```
# v
fig , ax = plt.subplots(2, 2, figsize=(12,8))
ax[0, 0].hist(df_college['Accept'] / df_college['Apps'] , bins=5)
ax[0, 0].set_title('Percentage Accepted')

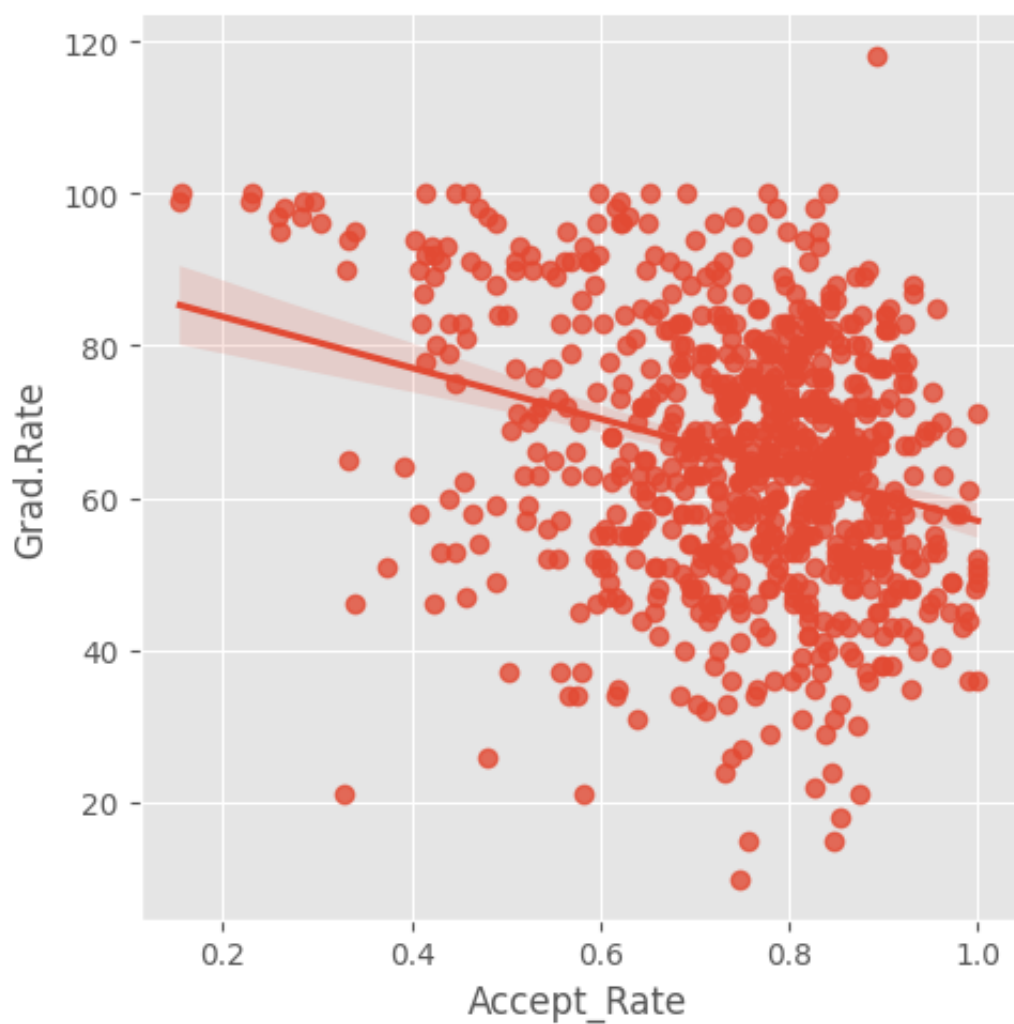
ax[0, 1].hist(df_college['Accept'] / df_college['Apps'] , bins=10)
ax[0, 1].set_title('Percentage Accepted')

ax[1, 0].hist(df_college['Accept'] / df_college['Apps'] , bins=15)
ax[1, 0].set_title('Percentage Accepted')

ax[1, 1].hist(df_college['Accept'] / df_college['Apps'] , bins=20)
ax[1, 1].set_title('Percentage Accepted');
```



```
# vi
# Acceptance rate and Graduation rate are negatively correlated
df_college['Accept_Rate'] = df_college['Accept'] / df_college['Apps']
sns.lmplot(x='Accept_Rate', y='Grad.Rate', data=df_college);
```



✓ Problem 9

```
df_auto = pd.read_csv('/content/drive/MyDrive/(SEM2)ML_DATA_FILES/auto.csv')
```

```
df_auto.info()
```

```
>>> <class 'pandas.core.frame.DataFrame'>
RangeIndex: 392 entries, 0 to 391
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   mpg                   392 non-null   float64
 1   cylinders              392 non-null   int64   
 2   displacement           392 non-null   float64
 3   horsepower             392 non-null   int64   
 4   weight                 392 non-null   int64   
 5   acceleration           392 non-null   float64
 6   year                   392 non-null   int64   
 7   origin                 392 non-null   int64   
 8   name                   392 non-null   object  
dtypes: float64(3), int64(5), object(1)
memory usage: 27.7+ KB
```

a) Quantitative vs Qualitative Predictors

Quantitative - mpg, cylinders, displacement, horsepower, weight, acceleration

Qualitative - year, origin, name

```
# b and c – get the range and std of each quantitative predictor
df_auto.describe()
```

```
>>>
```

	mpg	cylinders	displacement	horsepower	weight	acceleration
count	392.000000	392.000000	392.000000	392.000000	392.000000	392.000000
mean	23.445918	5.471939	194.411990	104.469388	2977.584184	15.5413
std	7.805007	1.705783	104.644004	38.491160	849.402560	2.7588
min	9.000000	3.000000	68.000000	46.000000	1613.000000	8.0000
25%	17.000000	4.000000	105.000000	75.000000	2225.250000	13.7750
50%	22.750000	4.000000	151.000000	93.500000	2803.500000	15.5000
75%	29.000000	8.000000	275.750000	126.000000	3614.750000	17.0250
max	46.600000	8.000000	455.000000	230.000000	5140.000000	24.8000

```
# d
pd.concat((df_auto.iloc[:10], df_auto.iloc[85:])).describe()
```

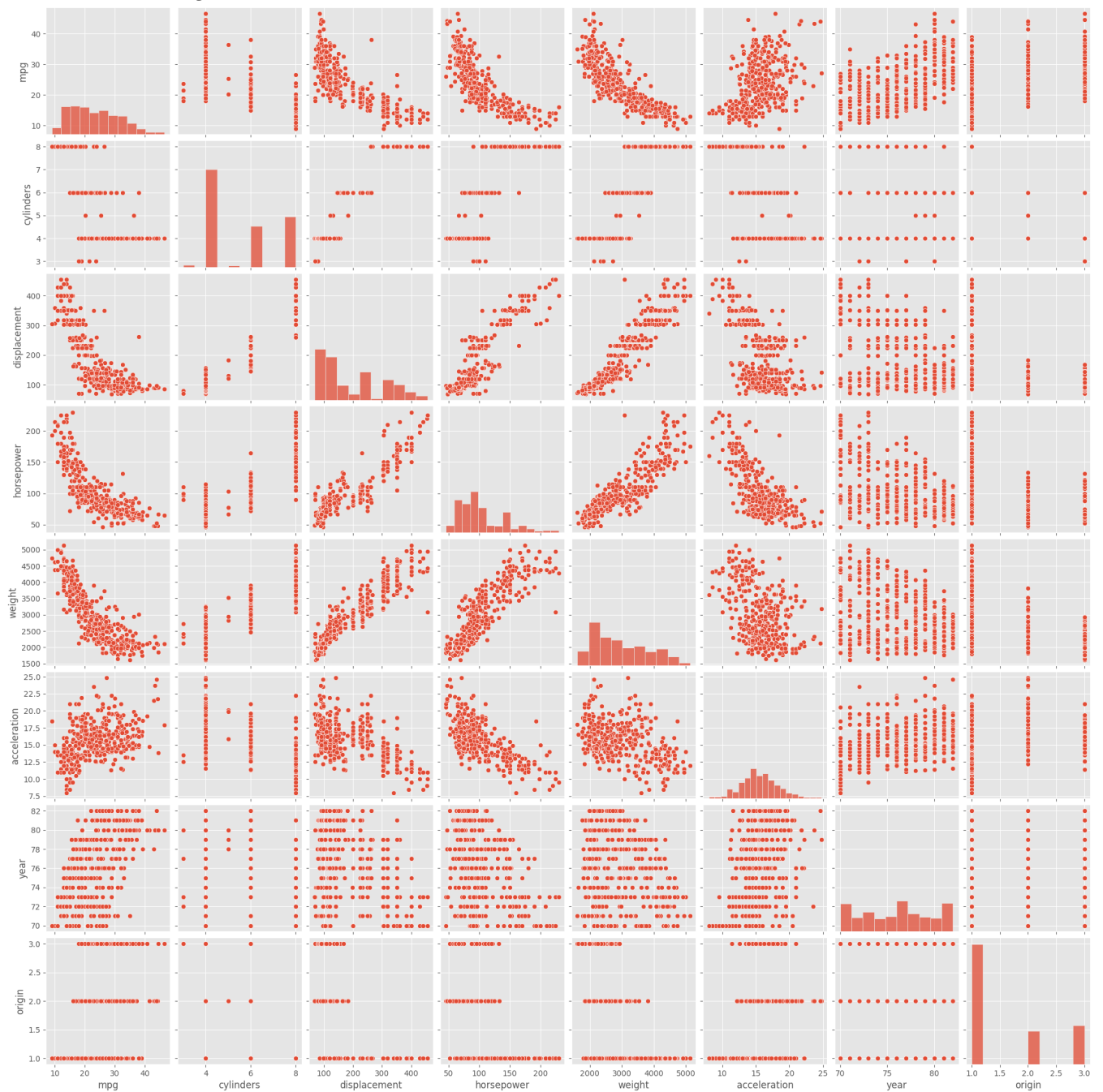


	mpg	cylinders	displacement	horsepower	weight	acceleration
count	317.000000	317.000000	317.000000	317.000000	317.000000	317.000000
mean	24.374763	5.381703	187.880126	101.003155	2938.854890	15.7041
std	7.872565	1.658135	100.169973	36.003208	811.640668	2.7199
min	11.000000	3.000000	68.000000	46.000000	1649.000000	8.5000
25%	18.000000	4.000000	101.000000	75.000000	2215.000000	14.0000
50%	23.900000	4.000000	146.000000	90.000000	2795.000000	15.5000
75%	30.500000	6.000000	250.000000	115.000000	3520.000000	17.3000
max	46.600000	8.000000	455.000000	230.000000	4997.000000	24.8000

```
# e
# Horsepower and displacement have a very strong positive linear relationship. Horsepower and
# have a strong negative relationship
sns.pairplot(df_auto)
```



<seaborn.axisgrid.PairGrid at 0x7e7214a47e10>



f) Looking at the pair plots above, mpg seems to have some relationship with just about all the other predictors.

✓ Problem 10

```
df_boston = pd.read_csv('data/boston.csv')
```

```
-----  
-  
FileNotFoundError                                Traceback (most recent call  
last)  
<ipython-input-83-0b63cbla5293> in <cell line: 0>()  
----> 1 df_boston = pd.read_csv('data/boston.csv')  
  
-----  
4 frames  
/usr/local/lib/python3.11/dist-packages/pandas/io/common.py in  
get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text,  
errors, storage_options)  
    871         if ioargs.encoding and "b" not in ioargs.mode:  
    872             # Encoding  
--> 873             handle = open(  
    874                 handle,  
    875                 ioargs.mode,
```

Next steps: [Explain error](#)

```
df_boston.shape
```

This data frame contains the following columns:

crim per capita crime rate by town.

zn proportion of residential land zoned for lots over 25,000 sq.ft.

indus proportion of non-retail business acres per town.

chas Charles River dummy variable (= 1 if tract bounds river; 0 otherwise).

nox nitrogen oxides concentration (parts per 10 million).

rm average number of rooms per dwelling.

age proportion of owner-occupied units built prior to 1940.

dis weighted mean of distances to five Boston employment centres.

rad index of accessibility to radial highways.

tax full-value property-tax rate per \$10,000.

ptratio pupil-teacher ratio by town.

black $1000(B_k - 0.63)^2$ where B_k is the proportion of blacks by town.

lstat lower status of the population (percent).

medv median value of owner-occupied homes in \$1000s.

```
# The pair plot is too large. Lets plot correlations and  
df_boston.corr()
```

c) There are no very strong relationships ($> .9$) with crime. The highest two are **rad**, **tax** and **lstat**. Crime is correlated with density of population (from what I've read before) so rad might be representative of how dense the population is if you are close to highways. Tax rates are generally higher the closer you are to a city center so higher tax rates might imply denser populations. And lower status (lstat) makes sense since more crime is committed by those less well off.

d) Below are the towns that have a max for each of the predictors. Seems there are some limitation in the data such that 132 towns have exactly 24 as a value for rad and 121 towns have exactly 396.9 as a value for black. Crime also seems to be strangely distributed with nearly all values hovering around 0 and a few several orders of magnitude greater.

```
df_boston.loc[df_boston.idxmax().unique()].style.highlight_max()
```

```
(df_boston['rad'] == 24).sum(), (df_boston['black'] == 396.9).sum())
```

```
plt.hist(df_boston['crim']);
```

```
# e
df_boston['chas'].sum()
```

```
# f
df_boston['ptratio'].median()
```



```
-----
-
NameError                                Traceback (most recent call
last)
<ipython-input-82-33d8dd2f4998> in <cell line: 0>()
      1 # f
----> 2 df_boston['ptratio'].median()
```

```
-----
NameError: name 'df_boston' is not defined
-----
```

Next steps: [Explain error](#)

```
# g
# rad and black are both those suspicious maximum values and crim is a ridiculous outlier.
# of bad data
df_boston.loc[df_boston['medv'].idxmin()]
```

```
# h
(df_boston['rm'] > 7).sum(), (df_boston['rm'] > 8).sum()
```

```
pd.DataFrame({'More than 8 rooms': df_boston[df_boston['rm'] > 8].describe().loc['50%'],
              '8 or less rooms' : df_boston[df_boston['rm'] <= 8].describe().loc['50%']})
```

Most of the predictors are the similar in both groups except crime and median value of homes - both about double for more than 8 rooms.

